# Challenge Clyvo — Olli Pet

## Apresentação do Trabalho

DevOps Tools & Cloud Computing — Sprint 4: Deploy automatizado com Azure DevOps

Turma 2TDSA — Time Olli Pet

| Integrante | RM |
|---|---|
| Andre Rosa Colombo | RM563112 |
| Gabriely Bonfim Silva | RM566242 |
| Henrique Rodrigues Vespasiano | RM562917 |
| Mirelly Sousa Alves | RM566299 |
| Ruan Luca Feliciano | RM562218 |

Este documento apresenta o que foi entregue na Sprint 4: a arquitetura com o fluxo de CI/CD, a configuração do projeto no Azure DevOps, a pipeline de build, testes e deploy, o banco de dados em nuvem e as evidências (prints e saídas de comando) de que a aplicação foi publicada automaticamente e persiste os dados no MySQL da Azure.

| | |
|---|---|
| Repositório GitHub | https://github.com/HenriqueRodriguesV/Sprint4DevOps |
| API em produção (Azure) | https://ollipet-api-rm562917.azurewebsites.net/swagger-ui.html |
| Projeto Azure DevOps | dev.azure.com/ollipetnoite - Sprint 4 – Azure DevOps (privado, Git, Scrum) |
| Pipeline | azure-pipelines.yml — CI (Build + testes + artefato) e CD (deploy) automáticos |
| Banco de dados | Azure Database for MySQL Flexible Server 8.0.21 - Mexico Central (schema ollipet) |

# 1. Descrição da solução

A Olli Pet é uma clínica veterinária que resolvia agendamento, vacinação, triagem de urgência e acompanhamento de tratamento por WhatsApp e planilhas. A API em Spring Boot é o backend do aplicativo mobile da clínica e concentra quatro regras de negócio:

- Triagem por questionário — classifica a urgência do pet a partir das respostas do tutor.
- Agendamento de consulta — controla a agenda dos veterinários e impede choque de horário.
- Carteira de vacinação — calcula sozinha o número da dose e a próxima aplicação.
- Tratamento em casa — transforma a prescrição em doses que o tutor confirma uma a uma.

### Benefícios para o negócio

A agenda deixa de depender da recepção conferir conflitos; a triagem funciona a qualquer hora; a clínica enxerga quem está com vacina atrasada antes de o tutor aparecer; e o veterinário passa a ver o percentual real de doses confirmadas. Com a Sprint 4, cada mudança no código chega à produção sozinha, testada, em minutos — sem deploy manual e sem risco de publicar uma versão que não compila ou quebra os testes.

# 2. Como cada requisito foi atendido

✔ Projeto no Azure DevOps "Sprint 4 – Azure DevOps": privado, Git, processo Scrum, professor convidado — seção 5.

✔ Pipeline de CI disparada a cada push na `master`: build com Maven, testes automatizados JUnit publicados e artefato .jar publicado — seção 6.

✔ Pipeline de CD disparada automaticamente após o artefato (dependsOn: Build), com deploy no Azure Web App — seção 6.

✔ Credenciais protegidas: usuário, senha e URL do banco em Variable Group com variáveis secret, nunca no código — seção 5.

✔ Banco de dados em nuvem (não H2): Azure Database for MySQL Flexible Server — seção 4.

✔ DDL comentado em arquivo separado: `script_bd.sql`, validado em MySQL 8 — seção 4.

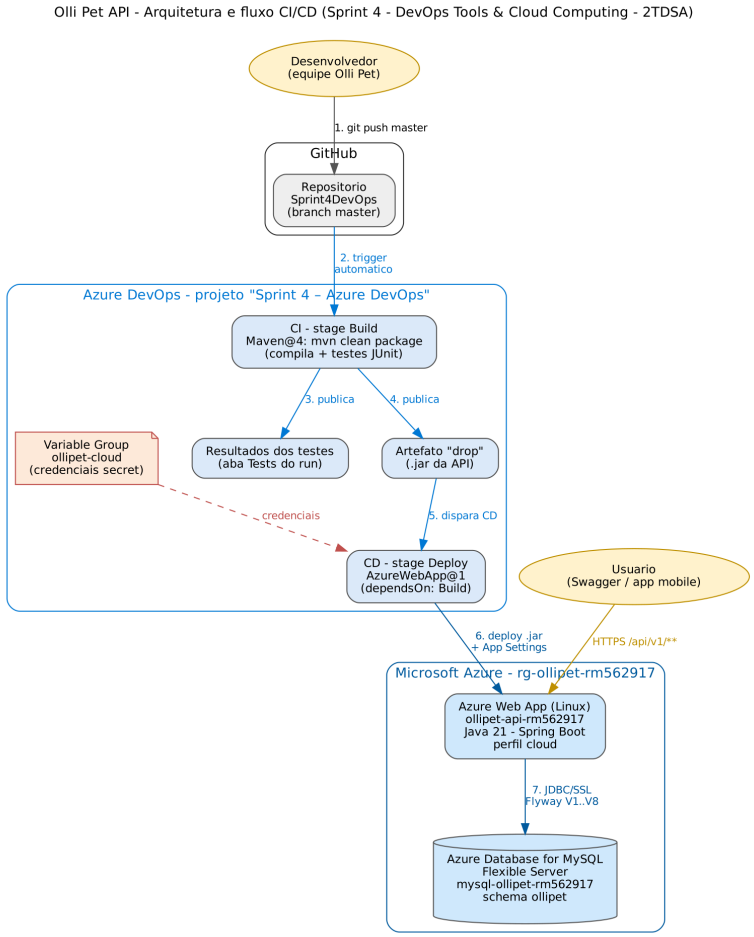
✔ Recursos criados via Azure CLI: `docs/azure/provisionar.sh` — seção 7.

✔ CRUD completo na tabela `pet` (relacionada a `responsavel`) com SELECT no banco após cada operação — seção 8.

✔ Desenho da arquitetura e do fluxo CI/CD e detalhamento dos componentes — seção 3.

✔ Código-fonte no GitHub com README de execução e deploy — seção 9.

# 3. Arquitetura e fluxo CI/CD




Fluxo: push na master → CI (Maven: build + testes) → artefato .jar → CD automático → Azure Web App → MySQL em nuvem.

1. O desenvolvedor faz git push na branch master do GitHub.
2. O push dispara a pipeline no Azure DevOps (trigger em master).
3. CI: mvn clean package compila e roda os testes JUnit; os resultados vão para a aba Tests.
4. O .jar é publicado como artefato drop.
5. CD: o stage Deploy começa sozinho quando o artefato existe.
6. A task AzureWebApp@1 aplica as App Settings (secrets do Variable Group) e publica o .jar.
7. No boot, a API conecta no MySQL via JDBC/SSL e o Flyway aplica as migrations V1..V8.

## Detalhamento dos componentes

| Componente | Nome do recurso | Tipo | Descrição funcional | Tecnologia |
|---|---|---|---|---|
| Repositório | Sprint4DevOps | Repositório Git | Código, pipeline e scripts | GitHub |
| Projeto DevOps | Sprint 4 – Azure DevOps | Projeto privado (Git, Scrum) | Boards, Pipelines, Library | Azure DevOps |
| Pipeline | azure-pipelines.yml | YAML multi-stage | CI (build/testes/artefato) e CD (deploy) | Azure Pipelines |
| Variable Group | ollipet-cloud | Library (secret) | URL, usuário e senha do banco | Azure Pipelines |

| Componente | Nome do recurso | Tipo | Descrição funcional | Tecnologia |
|---|---|---|---|---|
| Service Connection | sc-ollipet-azure | Azure Resource Manager | Permite à pipeline publicar na Azure | Azure DevOps |
| Resource Group | rg-ollipet-rm562917 | Grupo de recursos | Agrupa os recursos da solução | Azure |
| App Service Plan | plan-ollipet-rm562917 | Linux B1 | Capacidade do Web App | Azure App Service |
| Web App | ollipet-api-rm562917 | Web App Linux Java 21 | Executa a API (perfil cloud) | Azure App Service |
| Banco de dados | mysql-ollipet-rm562917 | MySQL Flexible Server 8.0.21 B1ms (Mexico Central) | Persistência relacional (schema ollipet) | MySQL + Flyway |

# 4. Banco de dados em nuvem

Localmente a API usa H2 em arquivo; na nuvem o perfil Spring cloud (`application-cloud.properties`) troca para o Azure Database for MySQL Flexible Server. O schema é criado exclusivamente pelo Flyway (ddl-auto=validate) e o módulo flyway-mysql foi adicionado ao pom.xml.

```
# src/main/resources/application-cloud.properties
spring.datasource.url=${DATABASE_URL}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.username=${DATASOURCE_USERNAME}
spring.datasource.password=${DATASOURCE_PASSWORD}
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.flyway.enabled=true
```

## Validação das migrations em MySQL 8

Antes de subir para a nuvem, as oito migrations foram aplicadas em um MySQL 8.0 limpo, na mesma ordem que o Flyway usa. Todas executaram sem erro e criaram as 18 tabelas do core com a carga inicial:

```
$ for f in V1 V2 V3 V4 V5 V6 V7 V8; do mysql ollipet < ${f}__*.sql; done
V1__criar_schema_base.sql                OK
V2__criar_agendamento_consulta.sql       OK
V3__criar_carteira_vacinacao.sql         OK
V4__inserir_dados_iniciais.sql           OK
V5__criar_triagem.sql                    OK
V6__inserir_protocolos_triagem.sql       OK
V7__criar_tratamento.sql                 OK
V8__criar_administrador.sql              OK

mysql> SELECT id_pet, nome, raca, id_resp FROM pet;
+--------+------+------------------+---------+
| id_pet | nome | raca             | id_resp |
+--------+------+------------------+---------+
|      1 | Thor | Golden Retriever |       3 |
|      2 | Nina | Siames           |       3 |
|      3 | Bud  | Poodle           |       4 |
+--------+------+------------------+---------+
```

O mesmo conteúdo, comentado, está em `script_bd.sql` na raiz do repositório (tabelas: usuario, responsavel, med_vet, administrador, pet, consulta, prontuario, vacina, aplicacao_vacina, queixa, pergunta_triagem, opcao_resposta, orientacao_triagem, triagem, resposta_triagem, observacao_triagem, tratamento, dose_tratamento).

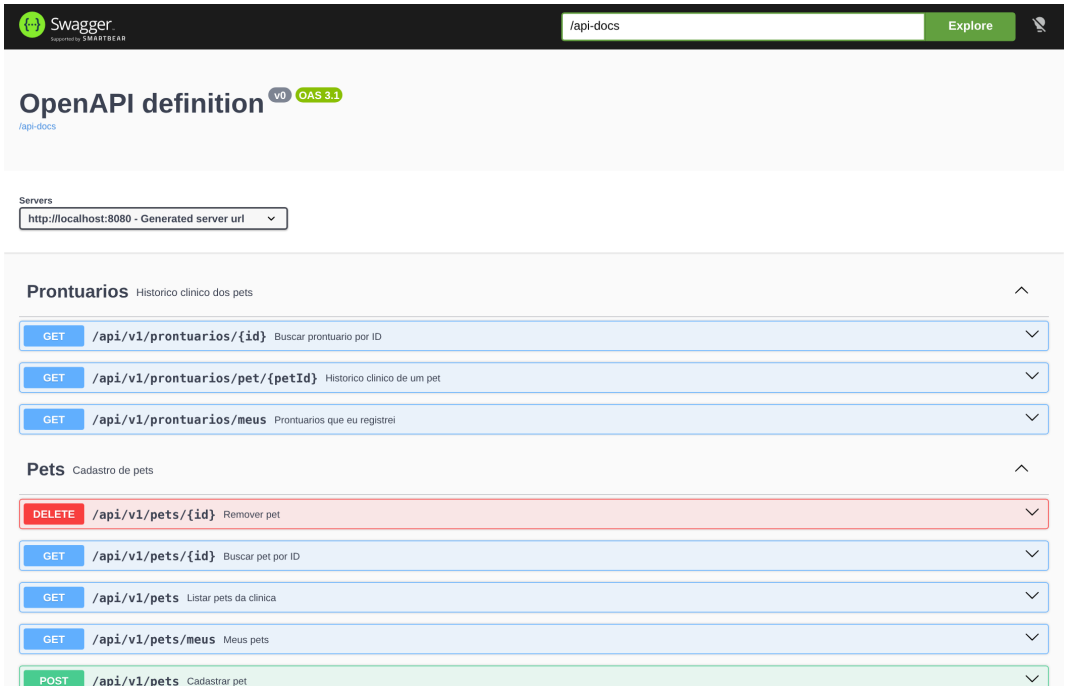
# 4.1 Validação local antes do deploy (perfil cloud + MySQL 8)

Antes de publicar, o mesmo build da pipeline foi executado e a API foi iniciada com o perfil cloud apontando para um MySQL 8.0: o Flyway aplicou as migrations V1..V8 e o Hibernate validou o schema (ddl-auto=validate) sem divergências.

## Testes automatizados (mvn clean package) — 30 testes, 0 falhas

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  ChallengeClyvoApplicationTests
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  model.StatusConsultaTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0  model.VacinaTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0  service.AvaliadorDeTriagemTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0  service.PlanoDeDosesTest


Flyway: Database: jdbc:mysql://localhost:3306/ollipet (MySQL 8.0)
Flyway: Successfully applied 8 migrations to schema `ollipet`, now at version v8
Started ChallengeClyvoApplication in 7.706 seconds
```




Swagger da API rodando com o perfil cloud (MySQL).

## CRUD completo com SELECT no MySQL após cada operação

```
$ POST /api/v1/auth/login  (maria.silva@email.com)
token: eyJhbGciOiJIUzI1NiJ9.eyJp...
```

```
$ POST /api/v1/pets
{
  "id": 4,
  "nome": "Bolinha",
  "descricao": "Vira-lata resgatado, muito brincalhao",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "idade": 2,
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
HTTP 201
mysql> SELECT id_pet, nome, raca, id_resp FROM pet ORDER BY id_pet DESC LIMIT 1;
+--------+---------+-----------+---------+
| id_pet | nome    | raca      | id_resp |
+--------+---------+-----------+---------+
|      4 | Bolinha | Vira-lata |       3 |
+--------+---------+-----------+---------+
```

```
$ GET /api/v1/pets/4
{
  "id": 4,
  "nome": "Bolinha",
  "descricao": "Vira-lata resgatado, muito brincalhao",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "idade": 2,
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
HTTP 200
```

```
$ PUT /api/v1/pets/4
{
  "id": 4,
  "nome": "Bolinha Castrado",
  "descricao": "Castrado em 10/2026",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "idade": 2,
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
HTTP 200
mysql> SELECT id_pet, nome, descricao FROM pet WHERE id_pet = 4;
+--------+-----------------+---------------------+
| id_pet | nome            | descricao           |
+--------+-----------------+---------------------+
|      4 | Bolinha Castrado | Castrado em 10/2026 |
+--------+-----------------+---------------------+
```

```
$ DELETE /api/v1/pets/4
HTTP 204
mysql> SELECT COUNT(*) FROM pet WHERE id_pet = 4;
+-------+
| total |
+-------+
|     0 |
+-------+
```

```
$ GET /api/v1/pets/4  (apos DELETE)
HTTP 404
```

# 5. Projeto no Azure DevOps

Projeto Sprint 4 – Azure DevOps na organização ollipetnoite, privado, com controle de versão Git e processo Scrum. O professor Felipe Oliveira (pf2128@fiap.com.br) foi convidado como Contributor do projeto, com nível de acesso Basic. URL, usuário e senha do banco ficam no Variable Group ollipet-cloud (Pipelines → Library), marcados como secret: a pipeline os injeta no Web App como App Settings no deploy, e eles não aparecem no código, no YAML nem nos logs. A Service Connection sc-ollipet-azure usa Workload Identity Federation (sem senha armazenada) e só tem permissão de Contributor no Resource Group do projeto.

## Configuração do projeto (item 4 do enunciado)

| Campo | Valor |
|---|---|
| Project Name | Sprint 4 – Azure DevOps |
| Description | Projeto para entrega da Sprint 4 do professor Felipe Cavalcante Vilella de Oliveira<br>Integrantes do grupo:<br>RM563112 - Andre Rosa Colombo - 2TDSA<br>RM566242 - Gabriely Bonfim Silva - 2TDSA<br>RM562917 - Henrique Rodrigues Vespasiano - 2TDSA<br>RM566299 - Mirelly Sousa Alves - 2TDSA<br>RM562218 - Ruan Luca Feliciano - 2TDSA |
| Visibility | Private |
| Version control | Git |
| Work item process | Scrum |

```
$ az devops project show -p "Sprint 4 – Azure DevOps"
Nome                    Visibilidade    Processo    Versionamento
----------------------  --------------  ----------  --------------
Sprint 4 – Azure DevOps  private         Scrum       Git
```

```
$ az devops user show --user pf2128@fiap.com.br
Nome                                 Email                            Acesso
-----------------------------------  -------------------------------  --------
Felipe Cavalcante Vilella de Oliveira  proffelipe.oliveira@fiap.com.br  Basic
```

```
$ az pipelines variable-group variable list --group-id 1   (ollipet-cloud)
Name                 Is Secret    Value
-------------------  -----------  -------
DATABASE_URL         True
DATASOURCE_USERNAME  True
DATASOURCE_PASSWORD  True
```

```
$ az devops service-endpoint list
Nome                Tipo     Autenticacao                Pronto
------------------  -------  --------------------------  --------
sc-ollipet-azure    azurerm  WorkloadIdentityFederation  True
HenriqueRodriguesV  GitHub   InstallationToken           True
```

```
$ az boards query (work items do projeto)
Estado     Titulo
---------  ------------------------------------------------------------------------------
Done       Provisionar recursos na Azure via CLI (RG, MySQL, App Service Plan, Web App)
Done       Migrar banco de H2 para Azure Database for MySQL (perfil cloud + flyway-mysql)
Done       Criar pipeline de CI (build Maven + testes JUnit + artefato)
Done       Criar pipeline de CD (deploy automatico no Azure Web App)
Done       Proteger credenciais do banco em Variable Group secret
Done       Documentar arquitetura, fluxo CI/CD e README
Committed  Gravar video da Sprint 4 com CI/CD e CRUD no banco em nuvem
```

# 6. Pipeline CI/CD

```
trigger:
  branches:
    include: [ master ]        # todo push na master dispara a pipeline

stages:
  - stage: Build               # CI
    jobs:
      - job: BuildAndTest
        steps:
          - task: Maven@4                    # mvn clean package = build + testes JUnit
            inputs: { goals: 'clean package', publishJUnitResults: true }
          - task: CopyFiles@2                # copia o .jar
          - task: PublishPipelineArtifact@1  # publica o artefato "drop"

  - stage: Deploy              # CD
    dependsOn: Build           # so comeca quando o artefato existe
    jobs:
      - deployment: DeployWebApp
        environment: producao-ollipet
        strategy: { runOnce: { deploy: { steps:
          - task: AzureWebApp@1              # App Settings (secrets) + deploy do .jar
        } } }
```

Arquivo completo: `azure-pipelines.yml` na raiz do repositório.

## Execução disparada pelo push (motivo individualCI)

O run abaixo nasceu sozinho de um git push: o CI compilou, rodou e publicou os 30 testes, publicou o artefato drop, e o CD começou automaticamente em seguida e publicou o .jar no Web App.

```
$ az pipelines runs show --id 4
Run         Motivo        Resultado    Branch           Commit   Autor
----------  ------------  -----------  ---------------  -------  -----
20261104.4  individualCI  succeeded    refs/heads/main  911a52a  Rique
```

```
Etapas do run (timeline):
STAGE CI - Build, testes e artefato            succeeded
   Initialize job                                        succeeded
   Checkout HenriqueRodriguesV/Sprint4DevOps@main to s   succeeded
   mvn clean package (compila + roda os testes JUnit)    succeeded
   Copiar o .jar executavel para a area de staging       succeeded
   Publicar artefato "drop" no Azure DevOps              succeeded
   Post-job: Checkout HenriqueRodriguesV/Sprint4DevOps@main t succeeded
   Finalize Job                                          succeeded
STAGE CD - Deploy no Azure Web App             succeeded
   Initialize job                                        succeeded
   Download Artifact                                     succeeded
   Aplicar App Settings e publicar o .jar                succeeded
   Finalize Job                                          succeeded
```

```
Testes publicados no run:
   Testes automatizados - Olli Pet API: total=30 aprovados=30
```

```
Artefatos publicados:
   drop (70610 KB)
```

# 7. Recursos na Azure (criados via Azure CLI)

Todos os recursos foram criados pelo script `docs/azure/provisionar.sh` (Azure CLI), na região Mexico Central — a assinatura Azure for Students só libera cinco regiões e o MySQL Flexible B1ms só tem SKU disponível em Mexico Central e Chile Central.

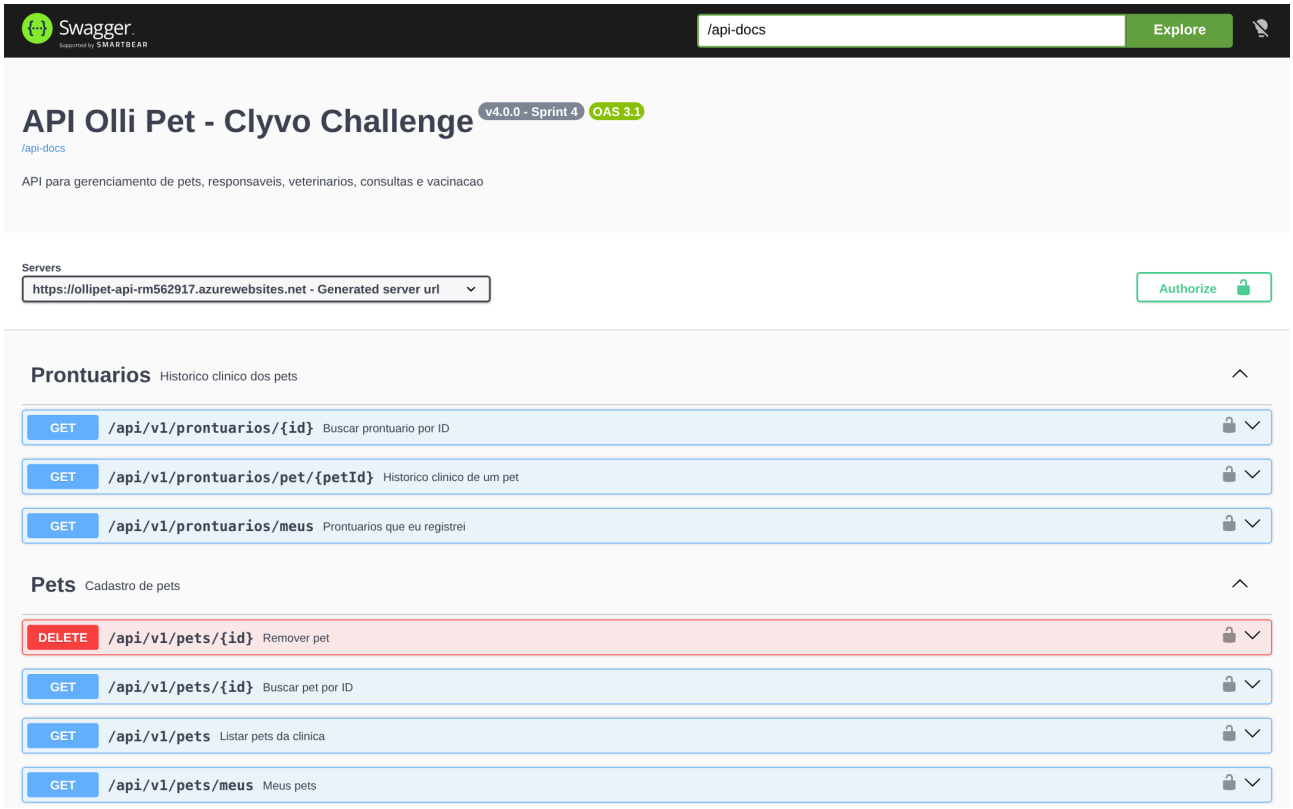
```
$ export MYSQL_ADMIN_PASSWORD='********'
$ bash docs/azure/provisionar.sh
>> 1/5 Resource Group: rg-ollipet-rm562917 (mexicocentral)
>> 2/5 Azure Database for MySQL Flexible Server: mysql-ollipet-rm562917
>> 3/5 Firewall do MySQL
>> 4/5 App Service Plan (Linux): plan-ollipet-rm562917
>> 5/5 Web App (Java 21): ollipet-api-rm562917
```

```
$ az resource list --resource-group rg-ollipet-rm562917 -o table
Name                    Location       Type                                  Status
----------------------  -------------  ------------------------------------  ---------
mysql-ollipet-rm562917  mexicocentral  Microsoft.DBforMySQL/flexibleServers  Succeeded
plan-ollipet-rm562917   mexicocentral  Microsoft.Web/serverFarms             Succeeded
ollipet-api-rm562917    mexicocentral  Microsoft.Web/sites                   Succeeded
```

```
$ az mysql flexible-server show -g rg-ollipet-rm562917 -n mysql-ollipet-rm562917
Estado    Versao    Sku            Tier       Host
--------  --------  -------------  ---------  ----------------------------------------------
Ready     8.0.21    Standard_B1ms  Burstable  mysql-ollipet-rm562917.mysql.database.azure.com
```

```
$ az webapp show -g rg-ollipet-rm562917 -n ollipet-api-rm562917
{
  "estado": "Running",
  "host": "ollipet-api-rm562917.azurewebsites.net",
  "plano": ".../serverfarms/plan-ollipet-rm562917",
  "runtime": "JAVA|21-java21"
}
```

```
$ az webapp config appsettings list (valores omitidos)
SPRING_PROFILES_ACTIVE
SERVER_PORT
WEBSITES_PORT
DATABASE_URL
DATASOURCE_USERNAME
DATASOURCE_PASSWORD
```




Swagger da API publicada no endereco azurewebsites.net.

# 8. CRUD completo no ambiente publicado

O CRUD foi executado na API publicada (https://ollipet-api-rm562917.azurewebsites.net) sobre a tabela pet, relacionada a responsavel pela chave estrangeira pet.id_resp. A listagem traz os pets da carga inicial (Thor, Nina), criados pelo Flyway no MySQL da Azure, junto com o registro novo — os dados vêm do banco em nuvem.

```
$ POST https://ollipet-api-rm562917.azurewebsites.net/api/v1/auth/login  (maria.silva@email.com)
HTTP 200 - token: eyJhbGciOiJIUzI1NiJ9...
```

```
$ POST /api/v1/pets   (CREATE)
{
  "id": 5,
  "nome": "Bolinha",
  "descricao": "Vira-lata resgatado, muito brincalhao",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "idade": 2,
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
HTTP 201
```

```
$ GET /api/v1/pets/5   (READ)
{
  "id": 5,
  "nome": "Bolinha",
  "descricao": "Vira-lata resgatado, muito brincalhao",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "idade": 2,
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
HTTP 200
```

```
$ PUT /api/v1/pets/5   (UPDATE)
{
  "id": 5,
  "nome": "Bolinha Castrado",
  "descricao": "Castrado em 11/2026",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "idade": 2,
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
HTTP 200
```

```
$ GET /api/v1/pets/meus   (pets da tutora, lidos do MySQL da Azure)
  id=5   Bolinha Castrado   Vira-lata          id_resp=3
  id=2   Nina               Siames             id_resp=3
  id=1   Thor               Golden Retriever   id_resp=3
```

```
$ DELETE /api/v1/pets/5   (DELETE)
HTTP 204
```

```
$ GET /api/v1/pets/5   (confirma remocao)
HTTP 404
```

### Conferência com SELECT direto no MySQL da Azure

Após cada operação, o SELECT abaixo é executado no banco em nuvem (mostrado também no vídeo):

```
-- depois do CREATE e do READ
SELECT id_pet, nome, raca, id_resp FROM pet ORDER BY id_pet DESC LIMIT 1;
-- depois do UPDATE
SELECT id_pet, nome, descricao FROM pet WHERE id_pet = {id};
-- depois do DELETE (0 = registro removido)
SELECT COUNT(*) FROM pet WHERE id_pet = {id};
```

## 9. Código no GitHub

Repositório público com o código da API, a pipeline, os scripts de provisionamento, o DDL e o README com o passo a passo de execução e deploy:
https://github.com/HenriqueRodriguesV/Sprint4DevOps

## 10. Vídeo de demonstração

O vídeo mostra a alteração no código, o push, o disparo automático do CI/CD com cada etapa explicada, o artefato, o deploy no Portal e o CRUD com SELECT no banco após cada operação.